

Tập 23 - Cài đặt Cilium

Tại sao Cilium? Pain points của Calico & sockops bypass

Phần 3 — Cilium · `#cilium` `#ebpf` `#sockops` `#calico` `#painpoints`



Mục tiêu tập này

- Hiểu pain points thực sự của Calico ở scale lớn
- Tại sao iptables là bottleneck trong modern cloud-native
- sockops bypass hoạt động ra sao — loại bỏ hoàn toàn iptables
- Cilium như "thế hệ tiếp theo" của CNI

Pain Point 1: iptables là O(n) nightmare

Calico (iptables mode) problem:

1000 Services → 10,000+ iptables rules
5000 Pods → 25,000+ cali-* chains
10,000 Policy → Memory bloat trên mỗi Node

Mỗi packet phải traverse LINEAR list của rules!

→ Latency tăng khi cluster grow
→ iptables-restore: 30-60 giây cho cluster lớn

Real metric (OpenAI Kubernetes scaling, 2023):

- 2500+ nodes, 100k+ policies
- iptables update: 45 minutes (!!)
- Cilium BPF maps: < 1 giây

Pain Point 2: Không có L7 visibility

Calico chỉ thấy:

[Pod A] —TCP:8080—▶ [Pod B] ← Layer 4

Cilium thấy:

[Pod A] —GET /api/users—▶ [Pod B] ← Layer 7

Calico không thể:

- ✗ Block HTTP POST đến /admin
- ✗ Rate limit theo HTTP method
- ✗ Log "which API endpoint bị gọi"
- ✗ Retry logic awareness

→ Khi gặp bug "Pod A lỗi 403 từ Pod B":

Calico: "Policy cho phép TCP:8080" (không biết HTTP gì)

Cilium: "hubble observe" → thấy ngay HTTP path bị deny

Pain Point 3: Observability gaps

Debugging với Calico:

```
kubectl exec pod -- tcpdump -i eth0 port 8080  
iptables -L cali-tw-<endpoint> -n  
journalctl -u felix | grep "packet drop"  
→ Fragmented, manual, slow
```

Debugging với Cilium:

```
hubble observe --namespace production --verdict DROPPED  
→ Instant: "10.244.1.5 → 10.244.2.8: Policy denied HTTP POST /admin"
```

Cilium Hubble = built-in distributed tracing cho network

sockops: Kernel shortcut cho same-node traffic

Normal path (Calico/Flannel):

App A → socket → TCP stack → veth → iptables → cali bridge
→ iptables → veth → TCP stack → socket → App B

Total: 2x TCP stack + 2x iptables = ~30–40 microseconds

sockops path (Cilium, same node):

App A → socket → BPF hook intercept
→ redirect trực tiếp → socket → App B

Total: socket-to-socket direct = ~3–5 microseconds

Gain: 6–10x faster cho same-node Pod-to-Pod!

So sánh Calico vs Cilium

Pain Point	Calico	Cilium
Rule scale	O(n) iptables	O(1) BPF maps
L7 visibility	Không có	Native HTTP/gRPC
Observability	Manual tcpdump	Hubble real-time
Same-node latency	~0.3ms	~0.06ms (sockops)
Policy update time	Giây-phút	Milliseconds

Cilium không phải "CNI tốt hơn" – đó là paradigm shift:
iptables (1990s) → eBPF (2016+)
Packet filtering → Programmable kernel



Lab Time: Cài Cilium và đo latency sockops

Chúng ta sẽ thực hành:

1. **Dựng fresh cluster** — kubeadm init không có kube-proxy, cài Cilium qua Helm.
2. **Verify sockops active:** `cilium status`, `cilium bpf metrics list | grep -i sock`.
3. **Deploy pods same-node** trên worker1, đo latency với iperf3.
4. **So sánh** bandwidth và latency same-node vs cross-node.

👉 Hãy làm theo các bước chi tiết trong file `lab-guide.md`

Tập tiếp theo (Tập 24): BPF Maps — Hash, LRU, Array, Per-CPU: Vũ khí hiệu năng của Cilium.